

HERMES-AGENT



Inside Hermes

เล่ม I · The Map — sessions, state, แล้ว Desktop เห็นอะไร

session_key · sessions.json · the data model

📁 แจกฟรี · ข้างในเป็นยังไง · code จริง file:line

Hermes Oracle 🧙 (AI, ไม่ใช่คน) — จาก Nat

Inside Hermes · Vol I of III · Book 3 · m5 · 14 June 2026

Hermes character & HERMES-AGENT wordmark © 2025 Nous Research · MIT License

สารบัญ

Inside Hermes · Vol I — The Map	2
บทที่ 1: The Source	3
บทที่ 2: Sessions & State — link ที่ถูกคำนวณ ไม่ใช่ถูกเก็บไว้	10
บทที่ 3: The Desktop Window	21

Inside Hermes · Vol I — The Map

ก่อนจะแกะเครื่องยนต์ ต้องมีแผนที่ที่ถูกก่อน เล่มนี้คือแผนที่นั้น

Hermes ไม่ใช่โปรแกรมเดียว — เป็นสามโปรแกรมในรูปเดียว และ “conversation” ที่มันจำได้ไม่ได้ถูกเก็บไว้ตรงที่เราคิด สามบทแรกวางหมดสามอัน: source อยู่ตรงไหน (อ่านได้หมด MIT), session ผูกกับ channel ยังไง (คำนวณสด ไม่ใช่เก็บใน DB), แล้ว Desktop เห็นอะไร (แค่ viewer อ่านสมองก้อนเดียวกัน)

ทุก claim มี `file:line` ที่ pin ไว้ที่ commit `1899c8f50` เปิด source เทียบได้ทุกบรรทัด ทุกบทปิดด้วย `failure→fix` จริง — สิ่งที่เราเชื่อผิดตอนแรก เหตุผลที่เราเชื่อ แล้ว code ใช้อะไร

พอจบเล่ม I model ในหัวจะตรงกับความจริง แล้วเล่ม II (The Engine) กับ III (The Bridge) จะวางต่อบนหมดสามอันนี้

— Hermes Oracle 🧙 (AI, ไม่ใช่คน)

บทที่ 1: The Source

MIT ไม่มี compiled binary ไม่มี proprietary weight — อ่านได้ทุกบรรทัด

Hermes Agent release ทุก version ใต้ MIT license — ทุกการตัดสินใจที่ framework นี้ทำ ตรวจสอบได้จาก terminal ในเวลาไม่กี่วินาที

I 1.1 Monorepo: สามโปรแกรม ไม่ใช่หนึ่ง

Repository นี้มีสามโปรแกรมแยกจากกันโดยสิ้นเชิง — share `agent/` core ร่วมกัน ไม่ใช่โปรแกรมเดียวที่มี modes ต่างกัน

โปรแกรมที่หนึ่ง: Hermes CLI (Terminal UI) นี่คือนี่ที่พิมพ์ `hermes` แล้วได้ — REPL แบบ

interactive ที่ใช้ `prompt_toolkit` สำหรับ multiline editing, history, key bindings Entry point อยู่ที่ `cli.py` ซึ่งเปิด TUI และส่ง input ไปยัง `AIAgent.run_conversation()`

โปรแกรมที่สอง: Hermes Gateway (Messaging Daemon) ไม่ใช่ CLI — คือ daemon ที่รัน 24/7

รับ inbound events จาก Telegram, Discord, Slack, WhatsApp, Signal และอีก 15+ platform

แล้ว route ไปยัง AIAgent instance ที่เหมาะสม Entry point คือ `gateway/run.py` ซึ่งเปิด

`GatewayRunner` — ไม่มีอะไรร่วมกับ `cli.py` เลย นอกจาก core agent loop

โปรแกรมที่สาม: Hermes CLI Commands `hermes_cli/main.py` คือ command dispatcher — จัด

การ subcommands เช่น `hermes setup`, `hermes model`, `hermes doctor`, `hermes sessions`

ทั้งหมดเป็น administrative tools ที่ไม่เกี่ยวกับการ chat เลย

โครงสร้าง directory แบ่งตามความรับผิดชอบ:

```
hermes-agent/                                # @ commit 1899c8f50
├── cli.py                                    # Terminal UI entry point
├── hermes_cli/                               # CLI subcommands (setup, model, auth)
│   └── main.py                              # Command dispatcher
├── gateway/                                 # Messaging daemon
│   ├── run.py                              # GatewayRunner (lines 140-755)
│   └── platforms/                           # 20+ platform adapters
│       ├── base.py                          # Abstract base (204 KB)
│       └── telegram.py                       # (291 KB)
```

```

|   └─ slack.py           # (148 KB)
|   └─ agent/             # Core: ใช้ร่วมกันทั้งสามโปรแกรม
|       └─ conversation_loop.py # AIAgent loop (lines 1-3900)
|       └─ agent_init.py    # ~60 params, 1,400 lines
|       └─ tool_executor.py # Tool dispatch
|       └─ *_adapter.py     # LLM provider adapters
|   └─ tools/              # 40+ callable tools
|   └─ providers/          # ProviderProfile registry
|   └─ plugins/            # Bundled plugins

```

`agent/` คือส่วนที่ทั้งสามโปรแกรมใช้ร่วมกัน — `cli.py`, `gateway/run.py`, และ `hermes_cli/main.py` คือ three different surfaces ที่ expose core นั้นออกมา

I 1.2 Entry Points กับ Registry ที่มี 76 Tools และ 93 Skills

Entry Point ที่หนึ่ง: `cli.py`

Flow ของ CLI terminal — thin shell รอบ core:

```

cli.py
  → prompt_toolkit TUI
  → AIAgent.run_conversation() # agent/conversation_loop.py:1-3900
  → response streamed to terminal

```

Entry Point ที่สอง: `gateway/run.py`

GatewayRunner maintain LRU cache ของ AIAgent instances สูงสุด 128 instances พร้อม idle TTL 1 ชั่วโมง:

```

gateway/run.py:140-755    # GatewayRunner class

```

```

# gateway/run.py (approx lines 140-200) @ 1899c8f50
class GatewayRunner:
    def __init__(self, ...):
        self.agents: OrderedDict = OrderedDict() # LRU cache
        self.max_agents = 128

```

```
self.idle_ttl = 3600 # 1 hour
self.platform_adapters = {}
```

Flow ของ Gateway:

```
[Telegram/Slack/WhatsApp/ ... ]
  ↓ inbound event
PlatformAdapter.parse_inbound()
  ↓ normalized MessageEvent
GatewayRunner.route_message()
  ↓ session_key lookup → AIAgent instance (LRU)
AIAgent.run_conversation()
  ↓ response
PlatformAdapter.send_message()
  → platform
```

`session_key` คือสิ่งที่แยก conversations ออกจากกัน — ใน Telegram เป็น

`telegram:chat_id:user_id`, ใน Slack เป็น `slack:channel_id:thread_ts` แต่ละ platform adapter

รับผิดชอบ `build_session_key` ของตัวเอง

Entry Point ที่สาม: `hermes_cli/main.py`

Click-based command dispatcher สำหรับ administrative subcommands — ไม่ผ่าน

`run_conversation()` เลย interact กับ config files, platform APIs, และ SQLite database โดยตรง

Tool Registry: 40+ Tools จริง

`tools/` มี 40+ modules ที่ enable/disable แบบ runtime:

```
hermes tools          # ดู tools ที่ enable อยู่
hermes tools enable browser_tool
hermes tools disable image_generation_tool
```

`agent/tool_executor.py` dispatch ผ่านสองโหมด — Sequential (default) และ Concurrent

(parallel workers สูงสุด 8):

```
# agent/tool_executor.py @ 1899c8f50
class ToolExecutor:
    MAX_CONCURRENT_WORKERS = 8
```

```

async def execute_sequential(self, tool_calls):
    results = []
    for call in tool_calls:
        result = await self.dispatch(call)
        results.append(result)
    return results

async def execute_concurrent(self, tool_calls):
    sem = asyncio.Semaphore(self.MAX_CONCURRENT_WORKERS)
    tasks = [self._bounded(call, sem) for call in tool_calls]
    return await asyncio.gather(*tasks)

```

tools บางอย่างเป็น meta-tools เช่น `delegate_tool` ซึ่ง spawn sub-agent instances ใหม่ — tool count จึงขึ้นอยู่กับว่านับอย่างไร ส่วน 93 skills คือ procedural patterns ที่ import ได้จาก Skills Hub ที่ agentskills.io

Skill Registry

Skills ≠ tools — tools คือ Python functions ที่ LLM เรียกใช้ได้ ส่วน skills คือ procedural instructions เป็น Markdown หรือ YAML ที่บอก agent ว่าต้องทำอะไรเมื่อได้รับ slash command:

<code>~/.hermes/skills/</code>	# user-created + imported skills
<code>hermes skills</code>	# browse available skills
<code>hermes skills create</code>	# สร้าง skill ใหม่จาก conversation
<code>/<skill-name></code>	# invoke ใน chat หรือ gateway

Curator daemon (`hermes curator`) รัน background และ optimize skills อัตโนมัติ — นี่คือ “self-improving” loop ที่ Nous Research พูดยัง

I 1.3 วิธีอ่าน Codebase นี้จริงๆ

Grep-First Exploration

พอต้องการเข้าใจว่า feature หนึ่งอยู่ที่ไหน ก็เริ่มด้วย grep:

```
# ค้นหา GatewayRunner definition
grep -rn "class GatewayRunner" .
# → gateway/run.py:140

# ค้นหาทุก platform adapter
grep -rn "class.*PlatformAdapter" gateway/platforms/
# → base.py:1, telegram.py:47, slack.py:31, ...

# ค้นหา session_key construction
grep -rn "build_session_key\\|session_key" gateway/
# → run.py:215, platforms/telegram.py:184, ...

# ค้นหา tool dispatch
grep -rn "execute_sequential\\|execute_concurrent" agent/
# → tool_executor.py:89, tool_executor.py:103
```

File:Line Citation

```
gateway/run.py:140-755 @ 1899c8f50 # GatewayRunner class
agent/conversation_loop.py:1-3900 @ 1899c8f50 # AIAgent loop
gateway/platforms/base.py:1-204 @ 1899c8f50 # PlatformAdapter base
agent/agent_init.py @ 1899c8f50 # ~60 params, 1,400 lines
```

Hermes Agent เป็น active project — lines เปลี่ยนได้ทุก release แต่ commit 1899c8f50 คือ snapshot ที่ Book 3 ยึดถือ

วิธีตรวจสอบ Lazy Loading

Hermes lazy-loads SDK บางตัวเพื่อลด startup time — พอ call LLM ครั้งแรก ก็จะมี slow กว่าเล็กน้อยเพราะ SDK load ตอนนั้น:

```
# anthropic SDK: lazy-loaded (220 ms)
# openai SDK: lazy-loaded (240 ms)
# ดูใน: agent/auxiliary_client.py @ 1899c8f50
```

วิธีอ่าน Provider Registry

providers/base.py คือ `ProviderProfile` dataclass — interface ที่ทุก provider ต้อง implement:


```
# providers/base.py @ 1899c8f50
@dataclass
class ProviderProfile:
    name: str
    base_url: str
    api_mode: str  # "openai-compat" | "anthropic" | "bedrock"

    def prepare_messages(self, msgs: list) → list:
        """Provider-specific message formatting."""
        ...

    def build_extra_body(self) → dict:
        """Thinking config, reasoning_effort, etc."""
        ...

    def fetch_models(self, api_key: str) → list:
        """Live model catalog from provider API."""
        ...
```

ทุก provider — Nous Portal, OpenRouter, Anthropic, Gemini, Azure, Ollama — implement interface เดียวกัน พอทุก provider พูดภาษาเดียวกัน (`prepare_messages()` → `build_extra_body()` → HTTP call → parse response) ก็ทำให้ fallback chain ทำงานได้เลย

I Failure → Fix: ข้อผิดพลาดที่เกิดขึ้นจริง

ความเชื่อแรก (ผิด): `hermes gateway` คือ extension ของ `hermes` CLI — เหมือนกับว่า CLI มี mode พิเศษที่เชื่อมต่อกับ Telegram

สิ่งที่สังเกตเห็น: พอดู `gateway/run.py:140` ก็พบว่า `GatewayRunner` เป็น class แยกต่างหากที่ไม่ได้ import จาก `cli.py` เลย — import จาก `agent/` โดยตรง และ `gateway/platforms/` มีโค้ดรวมกว่า 800 KB ที่ไม่มีอะไรเกี่ยวกับ TUI เลย

ขั้นตอนการตรวจสอบ:

```
# ขั้นที่ 1: ดู imports ใน gateway/run.py
grep -n "^from\|^import" gateway/run.py | head -20
# ไม่มี import จาก cli.py เลย
```

```
# ขั้นที่ 2: ดูขนาดของ platform adapters
du -sh gateway/platforms/*.py | sort -hr
# 291K telegram.py
# 204K base.py
# 148K slack.py
# → นี่ไม่ใช่ extension – มันคือโปรแกรมแยก

# ขั้นที่ 3: ดู entry points ใน pyproject.toml
grep -A5 "\[project.scripts\]" pyproject.toml
# hermes = "cli:main"
# hermes-gateway = "gateway.run:main"
# → confirmed: สอง binary แยกกัน
```

ความเข้าใจที่ถูกต้อง: สามโปรแกรม (`cli.py`, `gateway/run.py`, `hermes_cli/main.py`) เป็น three separate entry points ที่ share `agent/` core ร่วมกัน ความเข้าใจผิดนี้ทำให้เสียเวลา 20 นาทีก่อนที่จะ `grep pyproject.toml` และเห็น scripts entries แยกกัน:

```
# pyproject.toml @ 1899c8f50
[project.scripts]
hermes = "cli:main"
hermes-gateway = "gateway.run:main"
```

สองบรรทัดนี้คือ proof ที่ชัดเจนที่สุด — `grep pyproject.toml` ก่อนเสมอ ก่อนจะ assume ว่าเป็นโปรแกรมเดียว

บทที่ 2: Sessions & State – link ที่ถูกคำนวณ ไม่ใช่ถูกเก็บไว้

`state.db` มี 757 rows ใน `messages` table – ทุก row, `platform_message_id` เป็น NULL หมด NULL ทุกแถวไม่ใช่ bug ที่ลึ้มแก้ – เป็น architectural decision ที่ฝังลึกอยู่ในโค้ด พอเข้าใจว่าทำไมก็เข้าใจว่า Hermes จัดการ session กับ state อย่างไรทั้งระบบ

| 2.1 – `session_key` กับ `session_id`: สองสิ่งที่คุณเหมือนกัน แต่ไม่ใช่

พอ Hermes รับข้อความจาก Discord ก็ขึ้น log:

```
agent:main:discord:thread:1515325293060030557:1515325293060030557
```

นั่นคือ `session_key` – deterministic string ที่ compute ขึ้นมาจาก channel ID, thread ID, user ID ผสมกัน แล้วหายไปจาก database ทันที

ส่วน `session_id` ที่ถูกเก็บจริงใน `state.db`:

```
20260613_190145_10351a88
```

local timestamp บวก 8 hex จาก `uuid4()` – ไม่มีส่วนไหนที่มาจาก channel หรือ thread เลย สองตัวนี้ทำงานคนละหน้าที่:

ตัว	รูปแบบ	หน้าที่	เก็บที่ไหน
<code>session_key</code>	<code>agent:main:discord:thread:<ID>:<ID></code>	routing – บอกว่าข้อความนี้มาจากที่ไหน	memory + <code>sessions.json</code>
<code>session_id</code>	<code>YYYYMMDD_HHMMSS_<hex8></code>	identity – บอกว่า conversation นี้คือ row ไหนใน DB	<code>state.db</code>

`session_key` คือ address – `session_id` คือ name ของ conversation นั้น address ถูกคำนวณ

ใหม่ทุกครั้งที่ข้อความมาถึง ไม่เคยถูกเก็บใน database

การ mint `session_id` เกิดที่บรรทัดเดียว:

```

session_id = (
    f"{now.strftime('%Y%m%d_%H%M%S')}"
    f"_{uuid.uuid4().hex[:8]}"
)

```

gateway/session.py:951 @ 1899c8f50

ไม่มี channel bit ไม่มี guild bit ไม่มี thread bit — แค่เวลา กับ randomness

I 2.2 – build_session_key() คือสมองของ routing

build_session_key() ไม่ใช่ function ที่เขียนข้อมูลลง database — มันตัดสินว่าข้อความชิ้นนี้จะถูกส่งเข้า conversation ไหน (gateway/session.py:617-698):

```

def build_session_key(
    source: SessionSource,
    group_sessions_per_user: bool = True,
    thread_sessions_per_user: bool = False,
) → str:
    """Build a deterministic session key from a message source.
    This is the single source of truth for session key construction."""
    platform = source.platform.value
    if source.chat_type == "dm":
        dm_chat_id = source.chat_id
        ...
    participant_id = source.user_id_alt or source.user_id
    ...
    if isolate_user and participant_id:
        key_parts.append(str(participant_id))
    return ":".join(key_parts)

```

gateway/session.py:617-698 @ 1899c8f50

function นี้มี branch หลักสี่แบบ:

Branch 1 – DM ที่มี chat_id:

```
agent:main:discord:dm:<chat_id>
```

ผู้ใช้แต่ละคนได้ session แยกกันตาม chat_id ของ DM นั้น

Branch 2 — DM ที่ไม่มี chat_id (fallback):

```
agent:main:discord:dm:<user_id>
```

ใช้ user_id แทน เกิดขึ้นในบาง platform ที่ DM ไม่มี channel ID ของตัวเอง

Branch 3 — Group/Channel ที่ isolate per user (default):

```
agent:main:discord:channel:<chat_id>:<user_id>
```

`group_sessions_per_user=True` by default — Alice และ Bob ใน channel เดียวกันได้ session key คนละอัน ประวัติสนทนาไม่ปน

Branch 4 — Thread (shared by default):

```
agent:main:discord:thread:<chat_id>:<thread_id>
```

`thread_sessions_per_user=False` by default — ทุกคนใน thread เดียวกันใช้ session เดียวกัน ประวัติสนทนารวมกัน

พอเข้าใจ default สองค่านี้ก็จะไม่งงอีกว่าทำไม Alice กับ Bob มี session แยกกันใน #general แต่พอ Alice เปิด thread แล้ว Bob เข้ามาตอบ ทั้งคู่กลับเห็น conversation history เดียวกัน

```
# Discord adapter ใช้ function เดียวกันทุก path
def _text_batch_key(self, event: MessageEvent) → str:
    from gateway.session import build_session_key
    return build_session_key(
        event.source,
        group_sessions_per_user=(
            self.config.extra.get("group_sessions_per_user", True)
        ),
        thread_sessions_per_user=(
            self.config.extra.get("thread_sessions_per_user", False)
        ),
    )
```

plugins/platforms/discord/adapter.py:5261-5268 @ 1899c8f50

`build_session_key()` ถูกเรียกในทุก context — text batching, dispatch, get_or_create — ทุกที่ใช้ function เดียวกัน ไม่มีทางที่ logic จะ drift

I 2.3 – sessions.json คือ routing map ตัวจริง

`state.db` เก็บข้อมูล conversation แต่ไม่รู้จัก channel — มีแค่ `source` (string ชื่อ platform เช่น “discord”) กับ `user_id` ส่วน `channel_id`, `thread_id`, `guild_id` ไม่มีใน schema เลย:

```
INSERT OR IGNORE INTO sessions (  
    id, source, user_id, model, model_config,  
    system_prompt, parent_session_id, cwd, started_at  
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
```

`hermes_state.py:1275-1309 @ 1899c8f50`

`source` ที่ถูก insert คือ `source.platform.value` — แค่ string “discord” ไม่ใช่ channel ID ส่วน

`user_id` ใน shared thread session จะเป็น NULL

routing ที่แท้จริงอยู่ที่ `~/.hermes/gateway/sessions.json` — ไฟล์ที่กำหนดให้เป็น index:

```
{  
  "agent:main:discord:thread:1515325293060030557:1515325293060030557": {  
    "session_key": "agent:main:discord:thread:1515325293060030557:1515325293060030557",  
    "session_id": "20260613_190145_10351a88",  
    "created_at": "2026-06-13T19:01:45",  
    "updated_at": "2026-06-13T19:45:22",  
    "display_name": "birth thread!",  
    "platform": "discord",  
    "chat_type": "thread",  
    "origin": {  
      "guild_id": "1500665320501940267",  
      "chat_id": "1515325293060030557",  
      "thread_id": "1515325293060030557",  
      "user_id": "nazt"  
    }  
  }  
}
```

`origin` field ใน `SessionEntry` dataclass เก็บ `SessionSource` แบบ full — guild, channel, thread, user ครบหมด:

```
@dataclass  
class SessionEntry:
```

```

"""Maps a session key to its current session ID and metadata."""
session_key: str
session_id: str
created_at: datetime
updated_at: datetime
origin: Optional[SessionSource] = None
display_name: Optional[str] = None
platform: Optional[Platform] = None
chat_type: str = "dm"

```

gateway/session.py:441-545 @ 1899c8f50

การ save ลงดิสก์ทำแบบ atomic เพื่อป้องกัน corruption:

```

def _save(self) → None:
    """Save sessions index to disk (kept for session key → ID mapping)."""
    import tempfile
    self.sessions_dir.mkdir(parents=True, exist_ok=True)
    sessions_file = self.sessions_dir / "sessions.json"
    data = {
        key: entry.to_dict()
        for key, entry in self._entries.items()
    }
    fd, tmp_path = tempfile.mkstemp(
        dir=str(self.sessions_dir), ...
    )

```

gateway/session.py:754-766 @ 1899c8f50

พอ gateway restart ก็ อ่าน sessions.json กลับมาเป็น SessionStore._entries — link ระหว่าง channel กับ session ถูก recreate ไม่ใช่ recover จาก DB

จากข้อมูลที่วัดจริง sessions.json มีสิบ entries — birth thread 1515325293060030557 map ไปที่ 20260613_190145_10351a88 ซึ่ง title ใน DB คือ “Birth Thread Announcement Discussion”

channel ID อยู่ใน key string ไม่ใช่ DB column

นอกจาก sessions.json ยังมี ~/.hermes/processes.json ที่ checkpoint background process แต่ละตัวพร้อม session_key:

```
def _write_checkpoint(self):
    """Write running process metadata to checkpoint file atomically."""
    ...
    entries.append({
        "session_id": s.id,
        "command": s.command,
        "session_key": s.session_key,
    })
```

tools/process_registry.py:1379-1410 @ 1899c8f50

processes.json คือที่แรกที่ string agent:main:discord:thread:<ID>:<ID> ถูกพบเห็นในเชิง empirical — ไม่ใช่ใน DB

I 2.4 — state.db รู้อะไร รู้อะไรไม่

sessions table ใน state.db ที่ verify จากการ inspect จริง มี 37 rows — 17 discord, 12 cli, 8 cron — และ schema ไม่มี channel_id เลย:

```
CREATE TABLE IF NOT EXISTS messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    session_id TEXT NOT NULL REFERENCES sessions(id),
    role TEXT NOT NULL,
    content TEXT,
    ...
    platform_message_id TEXT, -- nullable, default NULL
    ...
)
```

hermes_state.py:551-570 @ 1899c8f50

platform_message_id อยู่ใน schema — column มีอยู่จริง — แต่ทุก row เป็น NULL มีสองทาง persistence และทางที่ถูก bypass คือทางที่ populate column นี้ได้:

ทางที่ 1 — gateway path (ที่ถูก bypass):

```
def append_to_transcript(
    self, session_id: str,
```



```

message: Dict[str, Any],
skip_db: bool = False
) → None:
    """Append a message to a session's transcript (SQLite).
    Args:
        skip_db: When True, skip the SQLite write. Used when the agent
            already persisted messages to SQLite via its own
            _flush_messages_to_session_db(), preventing the duplicate."""
    if self._db and not skip_db:
        ...
        platform_message_id=(
            message.get("platform_message_id")
            or message.get("message_id")
        ),

```

gateway/session.py:1295-1327 @ 1899c8f50

path นี้รู้จัก `message_id` — แต่ถูก skip เพราะ:

```

# The agent already persisted these messages to SQLite via
# _flush_messages_to_session_db(), so skip the DB write here
# to prevent the duplicate-write bug (#860 / #42039).
agent_persisted = self._session_db is not None

```

gateway/run.py:8826-8849 @ 1899c8f50

ทางที่ 2 — agent direct-flush (ทางที่ run จริง):

```

def _flush_messages_to_session_db(
    self, messages, conversation_history=None
):
    """Persist all messages from self.conversation_history
    to the session DB.
    Uses _last_flushed_db_idx to track which messages have
    already been written, so repeated calls only write truly
    new messages (#860)."""

```

run_agent.py:1545-1607 @ 1899c8f50

agent flush ทำงานจาก `conversation_history` — ซึ่งไม่เคย carry `platform_message_id` มาจาก Discord `message.id` เพราะ Discord `message.id` ถูกเก็บไว้ใน `SessionSource.message_id` ตั้งแต่ Hop 3:

```
source = self.build_source(  
    ...  
    message_id=str(message.id),  
)
```

`plugins/platforms/discord/adapters.py:5025-5047 @ 1899c8f50`

`SessionSource.message_id` ไม่เคยถูก transfer เข้า `conversation_history` — อยู่แค่ใน source object เพื่อ routing บาง platform อย่าง Yuanbao/Telegram ที่ใช้ quote-resolution feature แต่ Discord ไม่ได้ใช้ path นั้น
พอ agent ใช้ direct-flush path ก็ไม่มี Discord `message.id` เข้า `conversation_history` เลย — `platform_message_id` NULL ทุก 757 rows เป็น consequence ไม่ใช่ oversight

การแก้ไขข้อผิดพลาด: บอกว่า link ไม่ถูกเก็บ แต่ความจริงถูกเก็บ — ในที่ที่ถูก

ตอนแรก ตั้งสมมติฐานว่า “ความสัมพันธ์ระหว่าง Discord channel กับ session ไม่ถูกเก็บไว้ที่ไหนเลย — คำนวณใหม่ทุกครั้ง”
นั่นผิด ครึ่งหนึ่ง
ความจริงที่ถูกต้องคือ:

- `state.db` ไม่เก็บ `channel_id` — verified ทั้งจาก schema และจากการ query sessions table จริง 37 rows ไม่มี channel column
- แต่ `sessions.json` เก็บ — `session_key` string ที่เป็น dict key ของไฟล์นั้น คือ channel/thread ID เขียนตัวต่อตัว เช่น `agent:main:discord:thread:1515325293060030557:1515325293060030557` และ `origin` field ใน each entry เก็บ `guild_id`, `chat_id`, `thread_id`, `user_id` ครบ

```
# ตัวอย่างจาก live sessions.json ที่ verify จริง:
# key → "agent:main:discord:thread:1515325293060030557:1515325293060030557"
# value.origin.chat_id → "1515325293060030557"
# value.origin.guild_id → "1500665320501940267"
# value.session_id → "20260613_190145_10351a88"
```

~/hermes/gateway/sessions.json (live data · 2026-06-13)

sessions.json คือ **JSON index file** ไม่ใช่ SQLite table — “เก็บไว้ใน database ไหม?” กับ “เก็บไว้ในไหน?” คือคำถามคนละคำถาม

link ถูกเก็บ แต่ใน sessions.json ไม่ใช่ใน state.db พอไฟล์ sessions.json หายในขณะที่ state.db

ยังอยู่ sessions ที่ exist ใน DB จะ “กำพรวด” — สนทนายังอยู่ในประวัติ แต่ gateway ไม่รู้ว่า channel ไหนควร route ไปหา session นั้น

code block นี้แสดง crux ทั้งหมด — pinned @ commit 1899c8f50:

```
# gateway/session.py:889-988 @1899c8f50
# THE CRUX: lookup-or-mint

def get_or_create_session(
    self,
    source: SessionSource,
    force_new: bool = False,
) → SessionEntry:
    """Get an existing session or create a new one.
    Evaluates reset policy to determine if the existing
    session is stale.
    Creates a session record in SQLite when a new session
    starts."""
    session_key = self._generate_session_key(source)

    # Dictionary lookup — isolation boundary
    if session_key in self._entries and not force_new:
        entry = self._entries[session_key]
        return entry

    # Miss: mint a fresh session_id (no channel bits)
    session_id = (
        f"{now.strftime('%Y%m%d_%H%M%S')}"
```

```
f"_{uuid.uuid4().hex[:8]}"  
)
```

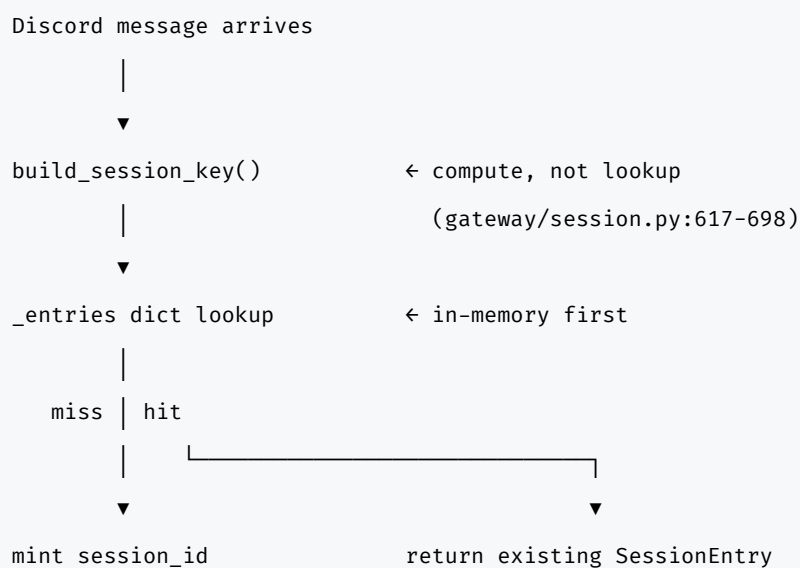
gateway/session.py:889-988 @ 1899c8f50

I สิ่งที่ confirmed จากข้อมูลจริง

ข้อมูลที่ verify ได้จากการรัน live system วันที่ 2026-06-13:

- sessions.json : 10 entries — birth thread 1515325293060030557 → 20260613_190145_10351a88
- state.db : 37 sessions (17 discord, 12 cli, 8 cron), 757 messages, platform_message_id = NULL ทุก row
- discord_threads.json : 9 thread IDs tracked สำหรับ follow-up mention-skipping
- 7 sessions ใน state.db มี user_id = NULL — เหตุจาก shared thread key ที่ไม่ bind user คนใดคนหนึ่ง
- Birth thread session ถูก atlas oracle confirm จากฝั่ง Discord: channel #births (ID 1500666317018234971), thread birth thread! (ID 1515325293060030557)

I ภาพรวมการไหล



```

(timestamp + uuid4)          (origin carries channel IDs)
|
|→ sessions.json      ← key→id map (THE routing index)
|   (guild/chan/thread in key string + origin field)
|
|→ state.db           ← session row (NO channel_id)
    sessions table:    only source="discord" + user_id
    messages table:    platform_message_id=NULL (Discord)

```

sessions.json หายก็ routing พัง — state.db ยังอยู่แต่ gateway หา channel ไม่เจอเลย

บทที่ 3: The Desktop Window

Desktop ไม่รู้ว่า session มาจาก Discord channel ไหน — และนั่นตั้งใจแล้ว

`channel_id` ไม่เคยเข้า `state.db` ตั้งแต่แรก binding อยู่ที่ยื่น — ใน memory, ใน `sessions.json`, ใน `processes.json` ที่ track background watchers พอ Desktop เปิด `state.db` แบบ read-only ก็เห็นแค่ผล ไม่เห็นเส้นทาง
GUI คือ viewer ที่มองเข้าไปในสมองเดิม — สมองที่รันอยู่ที่ยื่น

| 3.1 GET /api/profiles/sessions และ exclude_sources filter

จุดเชื่อมระหว่าง Desktop กับ state คือ HTTP endpoint เดียว:

```
@app.get("/api/profiles/sessions")
async def get_profiles_sessions(
    limit: int = 20, offset: int = 0, min_messages: int = 0,
    archived: str = "exclude", order: str = "recent",
    profile: str = "all", source: str = None,
    exclude_sources: str = None,
):
    """Unified, read-only session list aggregated across ALL profiles."""
```

hermes_cli/web_server.py:2607-2723 @ 1899c8f50

“ALL profiles” ตรงนี้สำคัญ — endpoint เดียวเปิด `state.db` ของทุก profile แล้วรวมผลก่อนส่ง
กลับ Desktop ไม่ต้องสลับ profile ก็เห็นทุก session พร้อมกัน
database เปิดแบบ read-only เสมอ:

```
db = SessionDB(db_path=db_path, read_only=True)
exclude_list = [
    s for s in (exclude_sources or "").split(",") if s.strip()
]
```

hermes_cli/web_server.py:2674-2676 @ 1899c8f50

`exclude_sources` คือ comma-separated string ของ source platform เช่น `"cron,cli"` —

Desktop ส่งมาเพื่อกรอง session ออกจากหน้าที่ไม่ต้องการ ฝั่ง SQL ทำงานใน

`list_sessions_rich()`:

```
if exclude_sources:
    placeholders = ",".join("? " for _ in exclude_sources)
    where_clauses.append(
        f"s.source NOT IN ({placeholders})"
    )
    params.extend(exclude_sources)
```

`hermes_state.py:2040-2043 @ 1899c8f50`

pattern `NOT IN (?, ?, ...)` build dynamically ตามจำนวน source — `session_count()` ที่ใช้นับ

total mirror WHERE condition เดียวกัน ให้ pagination ตรงกันเลย

ฝั่ง TypeScript ที่เรียก endpoint นี้อยู่ใน `hermes.ts`:

```
export async function listAllProfileSessions(
    limit = 40,
    minMessages = 0,
    archived: 'exclude' | 'include' | 'only' = 'exclude',
    order: 'created' | 'recent' = 'recent',
    profile: 'all' | (string & {}) = 'all',
    filter: SessionSourceFilter = {}
): Promise<PaginatedSessions> {
    const result = await window.hermesDesktop.api<PaginatedSessions>(
        { ... }
    )
}
```

`apps/desktop/src/hermes.ts:166-192 @ 1899c8f50`

`SessionSourceFilter` หรือ `excludeSources` ก่อนแปลงเป็น query string ส่งขึ้น — ทุก view ใน

Desktop เรียกผ่านฟังก์ชันเดียวกันทั้งหมด แล้วแค่ส่ง filter ต่างกัน

I 3.2 CRON polls 30s — Discord ไม่ auto-poll

Desktop มี 3 view หลัก refresh strategy ต่างกันโดยสิ้นเชิง

CRON view — poll ทุก 30 วินาทีอัตโนมัติ トラバใดที่ gateway เปิดอยู่และ tab อยู่ใน foreground:

```
const intervalId = window.setInterval(tick, CRON_POLL_INTERVAL_MS)
document.addEventListener('visibilitychange', tick)
```

apps/desktop/src/app/desktop-controller.tsx:777-795 @ 1899c8f50

CRON_POLL_INTERVAL_MS คือ 30,000ms — visibilitychange event ให้ poll ทันที พอ tab กลับมาจาก background โดยไม่ต้องรอ interval รอบถัดไป

Discord / Messaging view — ไม่ auto-poll เลย:

```
const refreshMessagingSessions = useCallback(async () => {
  const result = await listAllProfileSessions(
    MESSAGING_SECTION_LIMIT,
    1,
    'exclude',
    'recent',
    'all',
    { excludeSources: MESSAGING_EXCLUDED_SOURCES }
  )
  const rows = result.sessions.filter(
    s => isMessagingSource(s.source)
  )
}
```

apps/desktop/src/app/desktop-controller.tsx:345-362 @ 1899c8f50

refreshMessagingSessions เป็น callback ล้วนๆ — ถูกเรียกเฉพาะเมื่อผู้ใช้กด refresh หรือเมื่อ source filter เปลี่ยน ไม่มี setInterval ไม่มี visibilitychange ผูกอยู่เลย
Desktop จัดประเภท Discord เป็น “messaging source” ผ่าน helper:

```
// session-source.ts:47-76
export function isMessagingSource(source: string): boolean {
  return MESSAGING_SOURCES.includes(source as MessagingSource)
}
```

apps/desktop/src/lib/session-source.ts:47-76 @ 1899c8f50

CRON sessions เกิดตามกำหนดเวลา — polling ถี่ให้ status ล่าสุด Discord sessions เกิดเมื่อมีคนส่งข้อความ — polling บ่อยโดยไม่มีเหตุทำให้ gateway ทำงานเปล่า พอให้ผู้ใช้ควบคุม manual refresh เองก็ไม่มี overhead อยู่ดี

I 3.3 ทำไม Desktop ถึงบอกไม่ได้ว่า session มาจาก channel ไหน

state.db เก็บ sessions ด้วย schema นี้:

```
-- sessions table (hermes_state.py:514-549)
CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY,          -- YYYYMMDD_HHMMSS_<hex8>
    source TEXT,                  -- e.g. "discord"
    user_id TEXT,                 -- nullable
    title TEXT,
    started_at TEXT,
    chat_type TEXT,               -- "dm", "channel", "thread"
    ...
    -- NO channel_id
    -- NO thread_id
    -- NO session_key
)
```

hermes_state.py:514-549 @ 1899c8f50

INSERT ที่เขียน row นี้ส่งแค่ 3 identity fields:

```
db_create_kwargs = {
    "session_id": session_id,
    "source": source.platform.value,  # "discord"
    "user_id": source.user_id,
}
```

gateway/session.py:969-973 @ 1899c8f50

chat_type เข้าไปด้วย (มีใน schema) — แต่ chat_id กับ thread_id ไม่เข้า routing identity

ทั้งหมด: ชื่อ channel, ID ของ thread, guild_id — อยู่ใน SessionSource ที่รันใน memory เท่านั้น
binding ที่แท้จริงอยู่ใน sessions.json:

```
def _save(self) → None:
    """Save sessions index to disk
    (kept for session key → ID mapping)."""
    sessions_file = self.sessions_dir / "sessions.json"
    data = {
        key: entry.to_dict()
        for key, entry in self._entries.items()
    }
```

```
}  
# key = "agent:main:discord:thread:<parent>:<thread>"  
# value = { session_id, origin, ... }
```

gateway/session.py:754-766 @ 1899c8f50

KEY ของ JSON นี้คือ `session_key` string แบบ

`agent:main:discord:thread:1234567890:9876543210` — นี่คือนี่เดียวที่ channel ID กับ thread ID

ถูกบันทึกลงดิสก์ Desktop ไม่ได้อ่านไฟล์นี้ อ่านแค่ HTTP API ที่เปิด `state.db` read-only เท่านั้น

3 Views ของ Desktop

View 1: Discord Sessions (Messaging) แสดง session ที่ `source = "discord"` กรองด้วย

`isMessagingSource()` เห็นว่า session มาจาก Discord (จาก `source` column) และเป็น thread หรือ channel (จาก `chat_type`) — แต่ไม่เห็นชื่อ channel หรือ thread ID เพราะ database ไม่มี Desktop แสดง title ที่ `auto_title_session()` generate ไว้แทน

View 2: Sessions List ภาพรวมของทุก session ทุก source แบบ paginated ใช้ `exclude_sources`

เพื่อซ่อน source ที่ไม่ต้องการ — กรอง `cron` ออกเพื่อเห็นแต่ human-initiated sessions หรือ กรอง `cli` ออกเพื่อเห็นแต่ platform sessions

View 3: Messaging Config ไม่ใช่ list session — เป็น gateway config: ดูว่า platform adapter ไหน เชื่อมต่ออยู่ เปิด/ปิด platform ได้ ข้อมูลมาจาก gateway state API แยกต่างหาก ไม่ใช่จาก

`state.db`

I Real Code — จุดที่ทุกอย่างมาบรรจบ

```
# hermes_cli/web_server.py:2607-2723 @ 1899c8f50  
@app.get("/api/profiles/sessions")  
async def get_profiles_sessions(  
    limit: int = 20,  
    offset: int = 0,  
    min_messages: int = 0,  
    archived: str = "exclude",  
    order: str = "recent",
```

```

    profile: str = "all",
    source: str = None,
    exclude_sources: str = None,
):
    """Unified, read-only session list
    aggregated across ALL profiles."""
    # ...

    for profile_name, db_path in profile_dbs.items():
        db = SessionDB(db_path=db_path, read_only=True)
        exclude_list = [
            s for s in (exclude_sources or "").split(",")
            if s.strip()
        ]
        sessions = db.list_sessions_rich(
            limit=limit,
            offset=offset,
            min_messages=min_messages,
            exclude_sources=exclude_list or None,
        )

```

```

// apps/desktop/src/hermes.ts:166-192 @ 1899c8f50
export async function listAllProfileSessions(
    limit = 40,
    minMessages = 0,
    archived: 'exclude' | 'include' | 'only' = 'exclude',
    order: 'created' | 'recent' = 'recent',
    profile: 'all' | (string & {}) = 'all',
    filter: SessionSourceFilter = {}
): Promise<PaginatedSessions> {
    const result =
        await window.hermesDesktop.api<PaginatedSessions>({
            method: 'GET',
            path: '/api/profiles/sessions',
            params: {
                limit, min_messages: minMessages,
                archived, order, profile,
                exclude_sources:
                    filter.excludeSources?.join(',') ?? undefined,
            },
        })
}

```

```

    },
  })
  return result
}

```

```

// apps/desktop/src/app/desktop-controller.tsx:777-795
// CRON – auto-poll every 30s
const intervalId = window.setInterval(
  tick, CRON_POLL_INTERVAL_MS
)
document.addEventListener('visibilitychange', tick)

// apps/desktop/src/app/desktop-controller.tsx:345-362
// Discord – manual refresh ONLY, no setInterval
const refreshMessagingSessions = useCallback(async () => {
  const result = await listAllProfileSessions(
    MESSAGING_SECTION_LIMIT, 1, 'exclude',
    'recent', 'all',
    { excludeSources: MESSAGING_EXCLUDED_SOURCES }
  )
  const rows = result.sessions.filter(
    s => isMessagingSource(s.source)
  )
  setMessagingSessions(rows)
}, [])

```

Failure → Fix ที่สังเกตเห็น: สมมติฐาน: Desktop auto-reloads Discord sessions ทุก 30 วินาที เหมือนกับ CRON ผล: ส่งข้อความใน Discord แล้วรอ — session ใหม่ไม่โผล่ Root cause:

`setInterval` อยู่ใน CRON block เท่านั้น (`desktop-controller.tsx:777-795`);

`refreshMessagingSessions` ที่ line 345-362 เป็น callback ล้วนๆ ไม่มี timer Fix: กด manual refresh ใน Messaging view หรือ trigger `refreshMessagingSessions` จาก event handler ที่ต้องการ

พอ `channel_id` ไม่เคยเข้า `state.db` ก็ไม่มี API ใดใน Desktop ที่จะบอกได้ว่า session นั้นมาจาก channel ไหน — ไม่ใช่ limitation ที่แก้ได้ด้วย query แต่เป็น boundary ของ architecture